

RFC：请求意见/技术提案

Request for Comments

万能产品小助手 (PM AI)

魏子政审阅

2026-05-06 / 版本 v0.2

文档类型	RFC (Request for Comments)
定位	请求意见/技术提案 – 发出来让大家看看，有没有坑
核心问题	技术方案行不行得通？有没有更好的选择？
前置文档	MRD (市场需求文档)
后续文档	TRD (技术需求文档)

背景

MRD 确定了市场方向，PRD 确定了功能范围。本 RFC 提出技术选型和架构方案，征集意见后形成 TRD。

技术栈提案

架构方案

前端架构

SPA 单页应用，分为管理后台 (admin) 和开发者门户 (portal) 两个区域。路由守卫做权限控制。API 调用统一封装 axios 实例，Pinia 管理共享状态。

后端架构

六层分层：routers (路由层) → services (业务层) → models (数据层) → schemas (模型层) → tasks (任务层) → utils (工具层)。全部 async/await。

安全扫描架构

三层扫描管道：上传 → Semgrep (静态代码) → ClamAV (恶意文件) → LLM 语义分析 (权限合理性判断) → 汇总报告驱动审批。

层	技术选型	理由
前端框架	Vue 3 + Element Plus	国内部署友好，Element Plus 组件丰富，Vue 学习曲线低
构建工具	Vite	开发体验好，HMR 快，生态成熟
状态管理	Pinia	Vue 3 官方推荐，TypeScript 友好
后端框架	FastAPI	Python 生态，async 原生支持，Swagger 自动生成
ORM	SQLAlchemy 2.0 (async)	功能全面，async 支持，Alembic 迁移集成
数据库	PostgreSQL 16	JSONB 灵活，关系查询强，生态成熟
搜索引擎	OpenSearch 2.x	全文 + 语义搜索，中文分词支持 (IK)
对象存储	MinIO	S3 兼容，私有部署，轻量
认证	JWT (python-jose + passlib)	无状态，适合微服务，无需额外组件
容器编排	Docker Compose	开发/部署统一，轻量够用

讨论点

编号	状态	讨论内容
R-001	已决定	前端选 Vue 3 而非 React/Next.js (部署友好 + 团队熟悉)
R-002	已决定	后端选 FastAPI 而非 Spring (轻量 + 开发效率)
R-003	已决定	认证选 JWT 而非 Keycloak (减少组件依赖)
R-004	已决定	编排选 Docker Compose 而非 K8s (开发阶段够用)
R-005	待拍板	LLM 语义分析用哪个模型？内网部署还是 API 调用？(需魏子政拍板)
R-006	待拍板	OpenSearch 中文分词方案？IK 分词器还是其他？(需魏子政拍板)
R-007	已决定	暂不需要消息队列，用 asyncio 后台任务替代 (MASTER D-001 已锁定)

风险点

- **OpenSearch 资源占用**：单节点模式下内存占用较高（建议 512MB-1GB）
- **LLM 扫描延迟**：语义分析依赖外部模型，可能成为审批流程瓶颈
- **MinIO 兼容性**：部分 S3 高级特性 MinIO 可能不完全支持
- **Vue 3 生态**：相比 React，部分 UI 组件库选择较少

文档一致性维护规则

当 RFC 与 MASTER 总需求文档内容不一致时：

1. 接口文本、技术细节等局部不一致：以 MASTER 为主
2. 涉及重大流程或业务流转路径：必须由魏子政拍板决定
3. 讨论点状态变更（如待讨论 → 已决定）：需 MASTER 或魏子政拍板确认后更新

更新记录

版本	日期	变更内容
v0.2	2026-05-06	对齐 MASTER v1.0: R-005/R-006 改为待拍板, R-007 改为已决定; 新增文档一致
v0.1	2026-05-06	初始版本